CS4710-001: Artificial Intelligence

Spring 2026 – Assignment 6 – Due 11:59pm, Friday, May 1st

Submission

In this assignment, you will implement your own neural network solver from scratch in Python. You are provided with skeleton code `neuralnet.py`, `layers.py`, `losses.py`, `classification.py`, and `DiscTest.ipynb`. When you are finished, submit the following through GradeScope:

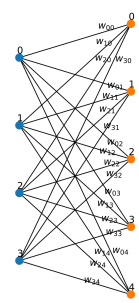
- `neuralnet.py`
- `layers.py`
- `classification.py` for the digit classification
- `hw6.pdf`: report file

You need to adhere to the honor code discussed in previous homeworks.

Problem Description: Backpropagation from Scratch

As we discussed in class, when we put a bunch of neurons together, we can learn arbitrarily complicated functions. So now we're going to use gradient descent to learn how to solve arbitrary fully connected feed forward networks using the *backpropagation* algorithm as a subroutine.

Forward Pass via Matrix Operations



At this point, we could just turn to one of the t libraries out there like PyTorch to optimize neural network models for us, but I want you to see if we use the right definitions, then creating a vanilla neural network solver really isn't that much code in numpy. It's also important that know how everything works under the hood when you run PyTorch.

The key trick is to recast a neural network as a sequence of matrix multiplications and element-wise applications of activation functions. Let's take the piece of a network on the right, for instance, where the blue nodes show inputs to the orange neurons:

Let's also assume that the i-th orange node has a bias b_i. Assuming that the activation function for the orange neurons is a function $f(u)$, then we could write the output of the ith neuron as:

$$a[i] = w_{0i}x_0 + w_{1i}x_1 + w_{2i}x_2 + w_{3i}x_3 + b_i$$
$$h[i] = f(a[i])$$

But there is a much more elegant way to write transformations for all inputs if we reformulate it as a matrix expression, and this will be much easier to implement in code. In particular, define the following

matrices

$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix}, \quad W = \begin{bmatrix} w_{00} & w_{10} & w_{20} & w_{30} \\ w_{01} & w_{11} & w_{21} & w_{31} \\ w_{02} & w_{12} & w_{22} & w_{32} \\ w_{03} & w_{13} & w_{23} & w_{33} \\ w_{04} & w_{14} & w_{24} & w_{34} \end{bmatrix}, \quad b = \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \end{bmatrix}$$

Then the output of a layer in the network can be defined in two stages

$$\begin{aligned} a &= Wx + b, \quad \text{which is a linear operation} \\ h &= f(a), \quad \text{which is a nonlinear operation applied element-wise to } a \end{aligned}$$

In general, the parameters to map from the output of a layer with N neurons to the input of a layer with M neurons can be described by an $M \times N$ weight matrix W and an Mx1 bias column vector b. To propagate information through the whole network, we continually apply this sequence of linear and nonlinear operators in matrix form. We just have to store the matrix W, the vector b, and the nonlinear function f that we're using at each layer. And that's it!

In sum, below is pseudocode that describes how to do a forward pass to transform the input from each layer to the next through the entire network

Algorithm 1 Forward Propagation Through a Fully Connected Feedforward Neural Network

```
function FORWARD(x)
    Let L be the number of layers
    h_0 ← x
    for k = 1, 2, ..., L do
        a_k = W_k h_{k-1} + b_k      ▷ Linear step. The input to this layer, h_{k-1}, is the output of the last layer
        h_k = f(a_k)                 ▷ Nonlinear step
    end for
    y_est ← h_L                      ▷ The output of the last layer is the output of our network
end function
```

Forward implementation in code I've implemented this for you already in `neuralnet.py` so you can see how this translates into numpy.

```python
def forward(self, x):
    """
    Do a forward pass on the network

    Parameters
    ----------
    x: ndarray(d)
        Input to feed through

    Returns
    -------
    ndarray(m)
        Output of the network
    """
    self.h = [x]
    self.a = [None]
    for k in range(len(self.Ws)):
```

```
        a = self.bs[k] + self.Ws[k].dot(self.h[k])
        h = self.fs[k]["f"](a)
        self.a.append(a)
        self.h.append(h)
    return self.h[-1].flatten()
```

Notice how I changed the indexing slightly, but that otherwise it's almost a straight translation from the pseudocode. I also remember the h's and the a's in member list variables, as they will come in handy in a moment.

Backward Pass via Matrix Operations

We're now ready to express the equations to compute the gradient over all parameters in the network. These will again boil down to some matrix equations that should be fairly painless to implement in code. Though a full derivation of these equations is beyond the scope of this writeup, I will give some intuition by looking at the at a simple network with 3 neurons, each with one input and one output, to avoid doing matrix derivatives for the moment. In particular, let's consider the following function:

$$g(x) = f_3\big(w_3 f_2(w_2 f_1(w_1 x + b_1) + b_2) + b_3\big)$$

In order to do updates, we need derivatives with respect to our weights w_1, w_2, w_3 and our biases b_1, b_2, b_3. Let us use the notation we established in the forward pass to define the following variables:

- $a_1 = w_1 x + b_1$
- $h_1 = f_1(a_1)$
- $a_2 = w_2 h_1 + b_2$
- $h_2 = f_2(a_2)$
- $a_3 = w_3 h_2 + b_3$
- $g(x) = f_3(a_3)$

Then we can compute the following derivatives from the outside of the expression inwards, using the chain rule (recall that a partial derivative holds all of the variables fixed as constants except for the one we're taking the derivative with respect to)

1. $$\frac{\partial g}{\partial w_3} = f_3'(a_3) \times h_2$$

2. $$\frac{\partial g}{\partial b_3} = f_3'(a_3)$$

3. $$\frac{\partial g}{\partial w_2} = f_3'(a_3) \times w_3 \times f_2'(a_2) \times h_1$$

4. $$\frac{\partial g}{\partial b_2} = f_3'(a_3) \times w_3 \times f_2'(a_2)$$

5. $$\frac{\partial g}{\partial w_1} = f_3'(a_3) \times w_3 \times f_2'(a_2) \times w_2 \times f_1'(a_1) \times x$$

6. $$\frac{\partial g}{\partial b_1} = f_3'(a_3) \times w_3 \times f_2'(a_2) \times w_2 \times f_1'(a_1)$$

A pattern is starting to emerge here. In particular, notice how equation 2 is contained in part of equations 3 and 4 and how equation 4 is contained in part of equations 5 and 6. So this means we'll be able to define some recursive substitutions from layer to layer as we go along, just like we remembered outputs of layers from one to the next as we went along during the forward pass.

From the point of view of the network, the first derivatives we're able to compute are with respect to parameters at the end of the network. We can then use these expressions to substitute in for parameters in layers that precede them. We can avoid recomputing things by remembering some of the products we computed along the way. This leads to an efficient dynamic programming algorithm known as backpropagation. It earns its name since, by contrast to evaluating moving forward layer to layer when evaluating an input, we actually start with the output of the network and compute the gradients backwards layer by layer.

There is one more thing I omitted, which is that we also define a loss function over the output, and we are really looking for the gradient with respect to the loss function. But the above gives a flavor for the patterns that emerge.

Below is the pseudocode for the matrix form of backpropagation for general feedforward networks of any shape. It may look daunting at first, but each bullet point should be a single line of code since we set up things in such an organized way with matrices.

Algorithm 2 Backpropagation Through a Fully Connected Feedforward Neural Network

Note: We write derivatives f' as `f_deriv` so the prime does not get lost in the shuffle
function BACKPROP(x, y)
    // After this iterates, the lists W_{derivs} and b_{derivs} will be populated with matrices that hold the derivatives of all weights and biases, respectively
    // x is input and y is ground truth label
    Let L be the number of layers
    Call FORWARD(x) to compute the a's and h's at each layer
    $y_{\text{est}} \leftarrow h_L$
    $g \leftarrow$ est_lossderiv(y_{est}, y) ▷ This is the derivative of the loss function with respect to the inputs of the last layer
    for $k = L, L-1, \ldots, 1$ **do**
        // Step 1: Propagate gradient backwards through the nonlinear output f_k of this layer
        if $k < L$ **then**
            $g \leftarrow g * f_{\text{deriv},k}(a_k)$ ▷ This is element-wise multiplication of g and $f_{\text{deriv},k}(a_k)$, which are parallel arrays

            // Step 2: Compute the gradients of the weights and biases at this layer
            $b_{\text{derivs}}[k] \leftarrow g$ ▷ We now have the gradient for biases in this level
            $W_{\text{derivs}}[k] \leftarrow g\, h_{k-1}^T$ ▷ This is a matrix multiplication. Treating h_{k-1} and g as column matrices, this performs their outer product to get the gradient for each weight at this layer

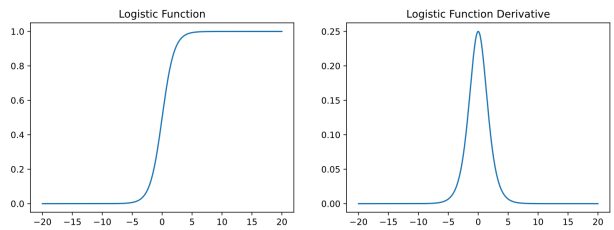
            // As a sanity dimension check, note that h_{k-1} is the output of the layer before, which is the input to this layer, and g is the gradient of the output of this layer. So if h_{k-1} is an $N \times 1$ matrix and g is an $M \times 1$ matrix, then gh_{k-1}^T will be an $M \times N$ matrix, which matches the dimensions of Wk. So each element of gh_{k-1}^T will hold the derivative of W_k's corresponding weight.
            // Step 3: Propagate the gradient backwards through the linear part of this layer to be used at the next layer back
            $g \leftarrow W_k^T g$ ▷ This is a matrix multiplication
        end if
    end for
end function

If yo look closely, you can match these steps up with the simple example I gave and see where the

substitutions happen from one layer to the next.

Problem 1: Leaky ReLU (15 points)

One of the issues with the logistic function (aka the "sigmoid function" in PyTorch parlance) is that it suffers from the problem of "vanishing gradients"; if the input to the logistic function is far from zero, then it is nearly flat, as shown below:



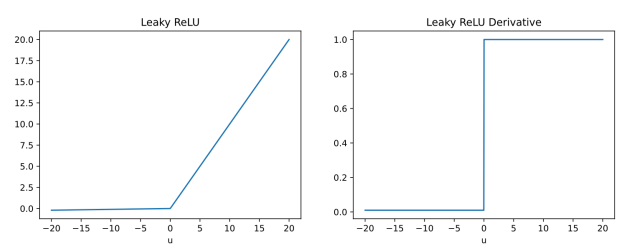
This makes it slow to learn, as the step sizes for internal weights in the network in these regimes will be very small. To address this, there's another activation function that's very popular known as the **Leaky Rectified Linear Unit (Leaky ReLU)**, which can be defined as the following piecewise function

$$f(u) = \begin{cases} u, & u > 0 \\ 0.01u, & u \leq 0 \end{cases}$$

The derivative of this function is then

$$f'(u) = \begin{cases} 1, & u > 0 \\ 0.01, & u \leq 0 \end{cases}$$

These functions are plotted below.



We've lost continuity of the derivative at the origin, but otherwise, it's great numerically and it never saturates, and you'll find that learning can happen much faster.

Your Task Fill in the methods `leaky_relu` and `leaky_relu_deriv` in the file `layers.py` to implement these functions. As you'll see in the method comments, your methods should take in numpy arrays and return the element-wise application of the leaky ReLU and its derivative at each element.

Problem 2: Backpropagation (30 points)

Now it's finally time to implement backpropagation.

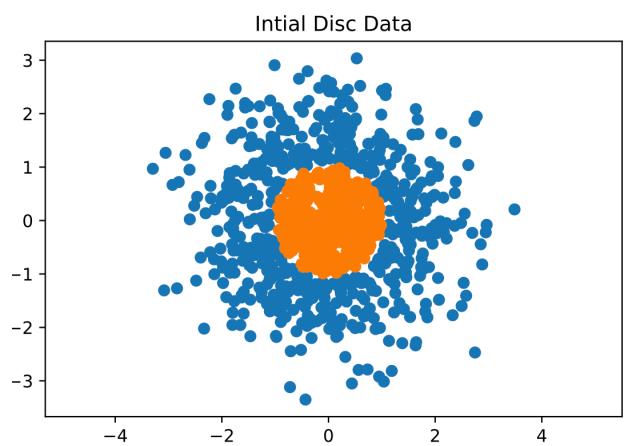
Your Task Study the class I've setup in `neuralnet.py`, and then fill in the method `backprop_descent`. This consists of two steps:

1. Use backpropagation to compute the gradients of all of the weights and biases;
2. Subtract the learning factor α times these gradients from all of the parameters.

The `backprop_descent` method accepts as input a single training example and its target output, and it updates the weights right away. This is referred to as *stochastic gradient descent*, and it contrasts to the *mini-batch gradient descent* where we waited until accumulating gradients from all training samples before updating the weights/bias.

As I mentioned above, I have already provided a method forward to evaluate each layer of the neural network and to store the inputs a to each activation function and the outputs h of the activation function. I have also provided a method `add_layer` to randomly initialize all of the weights and biases (this simply boils down to calling numpy's `randn` to generate random matrices). So once you finish `backprop_descent`, you'll have a complete system to train arbitrary feedforward neural networks.

To help you test your system, I've provided the notebook `DiscTest.ipynb` which will learn a network to separate the points on the inside of a disc from the points on the outside of the disc, which is not something that we could do with 2D logistic regression! Dataset shown below:



To do the separation, we use a network with the following three layers:

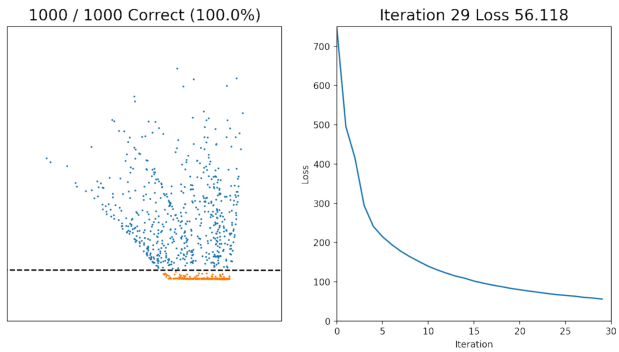
1. 100 neurons with a leaky ReLU
2. 2 neurons with a leaky ReLU
3. The final output neuron with a logistic activation, using the logistic loss

Using our little API, we can define this network with the following code

```
nn = NeuralNet(2, logistic_est_loss_deriv) # Input is in 2 dimensions, and we want to use logistic loss
nn.add_layer(100,  leaky_relu, leaky_relu_deriv) # First layer is 100 dimensions with a leaky ReLU
nn.add_layer(2, leaky_relu, leaky_relu_deriv) # Second layer is 2 dimensions with a leaky ReLU
nn.add_layer(1, logistic, None) # Last layer is the logistic function. Its derivative is handled separately
```

Then, since our API is designed to update one example at a time with backpropagation, we continuously feed it random permutations of our data one at a time. This is all in the notebook already and you can just run it. If it works properly, you should see an animation, where the left plot shows the 2 coordinates of the output of layer 2 as well as the linear separator inferred from the final layer, and the right plot shows the

loss over time. As you can see, after a few iterations, the orange points get moved to the other side of the final linear separator from the blue points



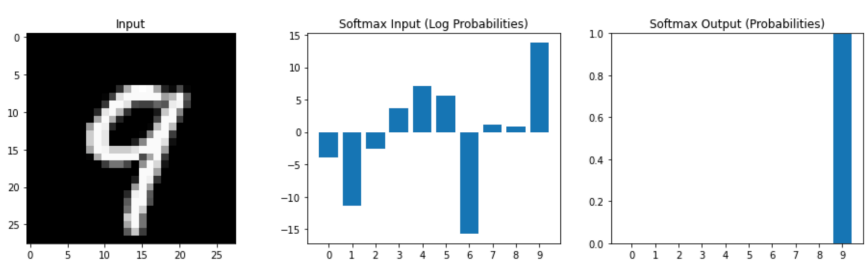
Problem 3: Softmax (15 Points)

Our code is definitely powerful enough to move beyond simple synthetic examples of 2-class problems, so let's move towards a more realistic application with MNIST digit classification to recognize 28×28 grayscale images of digits from 0-9. What's different about this problem from any that we saw so far is that it's a multiclass regression problem; our network needs to indicate which of 10 possibilities is the most likely. To do this, we'll have the output layer of our neural network consist of 10 neurons instead of just one. We can think of each neuron as indicating the probability that the input belonged to a particular digit.

To make these proper probabilities and to lead to a differentiable loss function, we're going to use a generalization of the logistic function to multi-class problems known as **softmax**. The softmax function is defined over an array of elements u as follows:

$$f(u)[i] = \frac{e^{u[i]}}{\sum_{j=1}^{N} e^{u[j]}}$$

Notice how the denominator makes it so that all of the elements of the softmax sum to 1. Furthermore, since we've exponentiated each term, the largest value will quickly dominate the smaller values, leading to a clearer winner in the output (which is really like an indicator function on the argmax of the function). In light of the above reasoning, we can view the input to the softmax as log probabilities, and the output of the softmax as ordinary probabilities. The example below shows the input and output of the softmax on the network you're about to train to classify the digits.



Your Task Fill in the `softmax` method in `layers.py` to complete code for the softmax activation layer. Be sure to adhere to the input/output spec in the docstring.

Problem 4: Digits Classification (40 Points)

We're now just about ready to apply this to digits! The last step is that we need to define a loss function over the output of the softmax, compared to ground truth. For this, define ground truth as an an "indicator array", or also referred to as a **one-hot vector**. Here are all of the possible 1-hot vectors based on ground truth

Ground truth value	1-Hot Vector y
0	$[1, 0, 0, 0, 0, 0, 0, 0, 0, 0]$
1	$[0, 1, 0, 0, 0, 0, 0, 0, 0, 0]$
2	$[0, 0, 1, 0, 0, 0, 0, 0, 0, 0]$
3	$[0, 0, 0, 1, 0, 0, 0, 0, 0, 0]$
4	$[0, 0, 0, 0, 1, 0, 0, 0, 0, 0]$
5	$[0, 0, 0, 0, 0, 1, 0, 0, 0, 0]$
6	$[0, 0, 0, 0, 0, 0, 1, 0, 0, 0]$
7	$[0, 0, 0, 0, 0, 0, 0, 1, 0, 0]$
8	$[0, 0, 0, 0, 0, 0, 0, 0, 1, 0]$
9	$[0, 0, 0, 0, 0, 0, 0, 0, 0, 1]$

Based on this, we can define a loss function which is a generalization of the logistic loss to multivariate output. Given the output yest of the softmax function and the ground truth 1-hot vectors y, we can define the **multivariate cross-entropy logistic loss** as

$$L(y, y_{\text{est}}) = -\sum_{i=1}^{N} y[i] \log\big(y_{\text{est}}[i]\big)$$

Your Task Put everything together in `classification.py` to train a neural network in our little API that classifies digits in the MNist dataset. Train a network using stochastic gradient descent on the training set for 60 iterations, and report the accuracy on all of the images in the test set. You should compare two networks: the first network should have a single hidden layer with 20 neurons and a softmax output, which you can define like this:

```
nn = NeuralNet(x_train.shape[1], softmax_est_crossentropy_deriv)
nn.add_layer(20, leaky_relu, leaky_relu_deriv)
nn.add_layer(10, softmax, None)
```

The second network should also start with a hidden layer with 20 neurons, but you should add an additional ReLU layer with 40 neurons.

Loading data PyTorch has this dataset built in, so we'll use it just for a moment to load it in:

```
import torch
from torchvision import datasets

imgres = 28
num_classes = 10

train_dataset = datasets.MNIST(root="./data", train=True, download=True)
test_dataset = datasets.MNIST(root="./data", train=False, download=True)

x_train = train_dataset.data.numpy().astype(float) / 255.0
x_test = test_dataset.data.numpy().astype(float) / 255.0
```

```
y_train = train_dataset.targets.numpy()
y_test = test_dataset.targets.numpy()

x_train = x_train.reshape(x_train.shape[0], imgres**2)
x_test = x_test.reshape(x_test.shape[0], imgres**2)

y_train = np.eye(num_classes)[y_train]
```

We want you to ...

In all your implementations, you should restrict them to use `numpy` library only. In your report, you should include for each problem:

1. A brief statement (∼paragraph) of what was done to answer the question (narratively explaining what you did in code to answer the question, at a high level).
2. A brief statement (∼paragraph) as to why this was done (why the question was answered in this way, not by doing something else. Some kind of rationale as to why you did x and not y or z to answer the question – why is what you did a suitable approach?).
3. A brief statement (∼paragraph) as to what was found. This should be as objective and specific as possible – just the results/facts. Do make sure to include numbers and a figure (=a graph or plot) in your statement, to substantiate and illustrate it, respectively.
4. A brief statement (∼paragraph) as to what you think the findings mean. This is your interpretation of your findings and should answer the original question.